



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 09

Problem Statement

Write a Python program to solve a multivariate linear regression problem using **Particle Swarm Optimization (PSO)**. The goal is to find the optimal weights (coefficients) for a linear model that predicts the median value of owner-occupied homes in the Boston Housing dataset by minimizing the Mean Squared Error (MSE) on the training data.

Background: Multivariate Linear Regression and the Boston Housing Dataset

Multivariate Linear Regression

Multivariate linear regression models the linear relationship between multiple independent variables (features) and a single dependent variable (target). The model is expressed as:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + \epsilon, \quad (1)$$

where y is the dependent variable (predicted value, e.g., median house value), $x_1, x_2, \dots, x_n$ are the independent variables (features), w_0 is the intercept (bias) term, $w_1, w_2, \dots, w_n$ are the coefficients (weights) for each feature to be optimized, and ϵ is the error term (difference between actual and predicted values). The objective is to determine the weights $w_0, w_1, \dots, w_n$ that minimize the Mean Squared Error (MSE) between predicted and actual values.

The Boston Housing Dataset

The Boston Housing dataset contains 506 instances of housing data from Boston suburbs, with 13 features and 1 target variable. It is available via OpenML. The features include: **CRIM**, per capita crime rate by town; **ZN**, proportion of residential land zoned for lots over 25,000 sq.ft.; **INDUS**, proportion of non-retail business acres per town; **CHAS**, Charles River dummy variable (1 if tract bounds river; 0 otherwise); **NOX**, nitric oxides concentration (parts per 10 million); **RM**, average number of rooms per dwelling; **AGE**, proportion of owner-occupied units built prior to 1940; **DIS**, weighted distances to five Boston employment centres; **RAD**, index of accessibility to radial highways; **TAX**, full-value property-tax rate per \$10,000; **PTRATIO**, pupil-teacher ratio by town; **B**, a demographic index by town; and **LSTAT**, percentage of lower-status population. The target variable is **MEDV**, median value of owner-occupied homes in \$1000s.

Algorithm Specifications

Implement a Particle Swarm Optimization (PSO) algorithm to optimize the weights of the linear regression model. PSO is a population-based optimization technique inspired by the social behaviour of birds flocking. Each particle represents a candidate solution (a set of weights) and moves through the search space to find the optimal weights.

1. **Representation:** A particle's position is represented by a 14-dimensional vector $\mathbf{x} = [w_0, w_1, w_2, \dots, w_{13}]$, where w_0 is the intercept term and w_1 to w_{13} are the coefficients for the 13 features. The prediction for a data point is given by $y_{\text{pred}} = w_0 + \sum_{j=1}^{13} w_j x_j$, where x_j are the (standardised) feature values.
2. **Fitness Function:** The goal is to minimize the Mean Squared Error (MSE) on the training data, defined as

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - y_{\text{pred},i})^2,$$

where N is the number of training data points, y_i is the actual target value for the i -th training point, and $y_{\text{pred},i} = w_0 + w_1 x_{1,i} + \dots + w_{13} x_{13,i}$ is the predicted value. A lower MSE indicates a better fit.

3. **Constraints:** Weights are bounded such that $-50 \leq w_j \leq 50$ for all $j = 0, \dots, 13$. Clamping is used to enforce these bounds: if a weight exceeds 50, it is set to 50; if it falls below -50 , it is set to -50 . *Note:* with standardised features, the intercept w_0 will be approximately equal to the mean of the target variable (≈ 22.5), and all feature weights will typically lie well within this range.
4. **Data Splitting and Feature Standardisation:** The dataset is split into 80% training and 20% testing sets using `random_state=42`. The features are then standardised using `StandardScaler`: the scaler is fitted on the training set only and applied to both the training and testing sets. This ensures all features have zero mean and unit variance, which keeps the optimal weight values within a compact range. Only the features X are standardised; the target y remains in its original units (\$1000s). The testing set is used only for the final evaluation.
5. **Initialization:** Set `np.random.seed(42)` before any PSO operations to ensure reproducibility. Each particle's position is then initialized uniformly at random within $[-50, 50]$ for each dimension. Velocities are initialized uniformly in $[-1, 1]$.
6. **Velocity and Position Updates:** Each particle's velocity and position are updated at each iteration using:

$$\mathbf{v}_i^{t+1} = w \cdot \mathbf{v}_i^t + c_1 \cdot r_1 \cdot (\mathbf{p}_{\text{best},i} - \mathbf{x}_i^t) + c_2 \cdot r_2 \cdot (\mathbf{g}_{\text{best}} - \mathbf{x}_i^t),$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1},$$

where $\mathbf{v}_i^t$ is the velocity of particle i at iteration t (14-dimensional), $\mathbf{x}_i^t$ is its position, w is the inertia weight, c_1 is the cognitive coefficient (pulls the particle toward its personal best $\mathbf{p}_{\text{best},i}$), c_2 is the social coefficient (pulls the particle toward the global best $\mathbf{g}_{\text{best}}$), and $r_1, r_2 \in [0, 1]$ are independent random scalars drawn uniformly at each update step.

Input Format

Read from standard input a single line: `w c1 c2 num_particles num_iterations` (space-separated), where w is the inertia weight (e.g., 0.7), c_1 is the cognitive coefficient (e.g., 1.5), c_2 is the social coefficient (e.g., 1.5), `num_particles` is the number of particles in the swarm (e.g., 50), and `num_iterations` is the number of iterations to run PSO (e.g., 2000).

Output Format

Print the following three lines to standard output:

- The best weights as $[w_0, w_1, \dots, w_{13}]$ — a list of **14** floats, each rounded to 3 decimal places.
- The training MSE — a float rounded to 2 decimal places.
- The testing MSE — a float rounded to 2 decimal places.

Sample Input

```
0.7 1.5 1.5 50 2000
```

Sample Output

```
Best Weights: [22.816, 0.032, 0.209, 0.288, 0.574, -0.44, 4.006, -0.497,  
-2.304, 0.81, -0.713, -1.853, 2.519, -4.034]
```

```
Training MSE: 24.69
```

```
Testing MSE: 32.65
```

Note: The sample output above was generated by running the reference solution with `np.random.seed(42)`. Minor numerical differences may arise across platforms or library versions due to floating-point arithmetic, but the values should be close to those shown.

Implementation Notes

Data Loading

Use `fetch_openml` to load the Boston Housing dataset from OpenML.

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# Load dataset
boston = fetch_openml(name='boston', version=1, as_frame=False)
X = boston.data.astype(float) # Features: (506, 13)
y = boston.target.astype(float) # Target: (506,)
```

```

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardise features (fit on training set only)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

Suggested Structure

```

# Load and split data; standardise features
# Set np.random.seed(42)
# Initialize particles: positions in [-50, 50], velocities in [-1, 1]
# Initialize personal bests (pbest) and global best (gbest)
# For each iteration:
#     For each particle:
#         Compute fitness (MSE on training data)
#         Update personal best if fitness improves
#         Update global best if fitness improves
#     Update velocities using inertia, cognitive, and social terms
#     Update positions and apply clamping to [-50, 50]
# Evaluate global best weights on training and testing data
# Output results

```